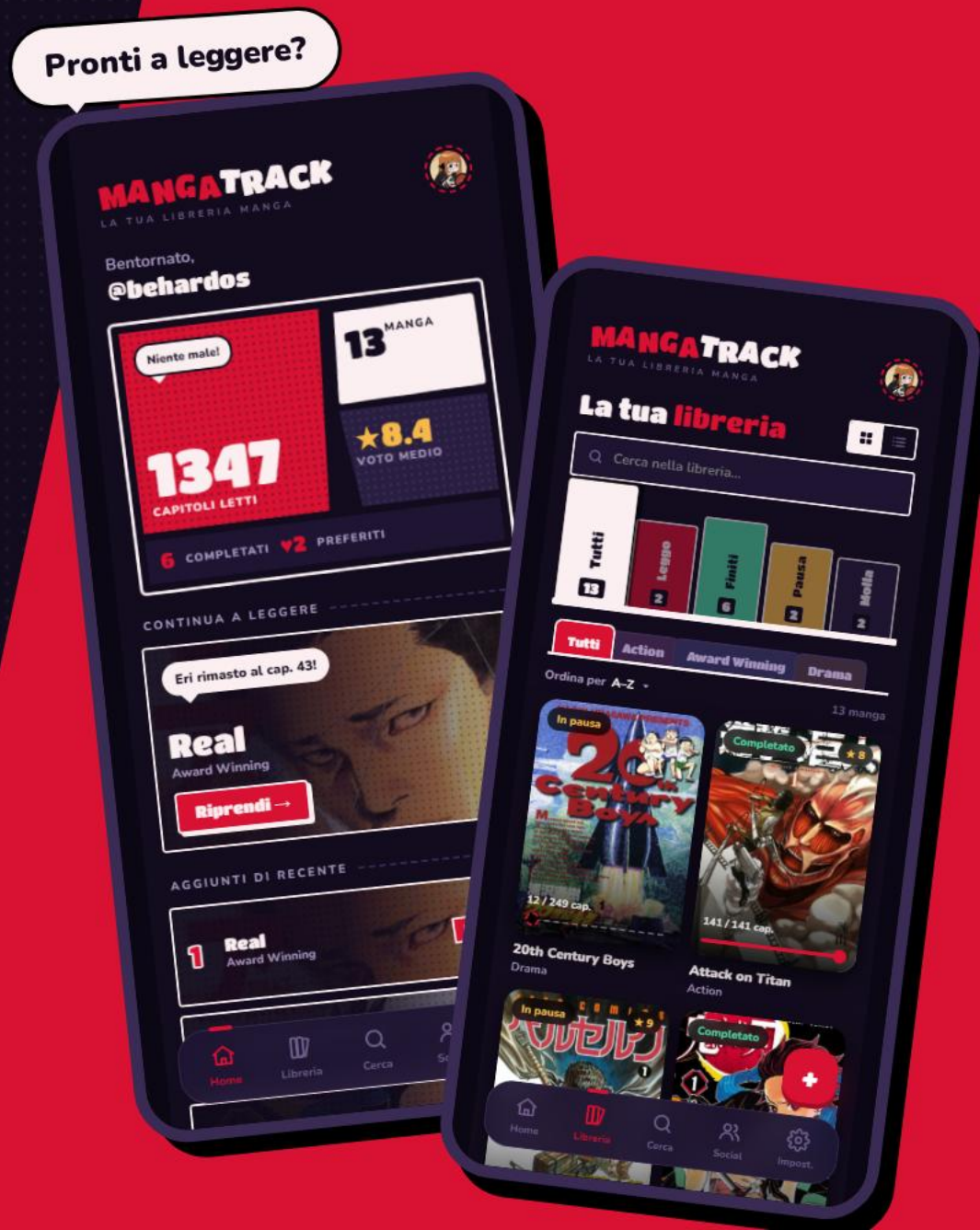
MANGATRACK

LA TUA LIBRERIA MANGA

Progetto d'esame
Behar Myzyri · 2026

React 18 | Vite 5 | Supabase | AniList GraphQL | Vercel



MANGATRACK
LA TUA LIBRERIA MANGA

01

Il problema, l'obiettivo

Note ovunque LETTURE SPARSE
Capitoli segnati tra note, screenshot e siti diversi, o affidati alla memoria.

Tracker pesanti TROPPO COMPLESSI
Gli strumenti esistenti sono pensati per desktop, in inglese e pieni di opzioni.

Niente amici ZERO SOCIAL
Difficile vedere cosa leggono gli amici o consigliare loro una serie.

OBIETTIVO **Un unico posto, semplice e da smartphone, per segnare ogni capitolo e condividere le letture.**



Il progetto in breve



MangaTrack è una single page application per tracciare i manga letti: capitoli, voto, recensione e scaffale di ogni serie.

COSA FA

Login e profilo · Libreria a 5 scaffali · Ricerca su AniList · Feed social, follow e consigli · Interfaccia IT / EN · Installabile su smartphone

Lo stack sullo scaffale

React 18

Componenti funzionali, hook e Context API. Virtual DOM per aggiornare solo ciò che cambia.

UI

Vite 5

Dev server con hot reload e build di produzione ottimizzata in dist/.

BUILD

Supabase

Backend as a service: Auth, PostgreSQL via REST e Storage per gli avatar.

BACKEND

AniList

API GraphQL per catalogo, copertine e trending.

DATI

Vercel

Hosting statico e deploy automatico da GitHub.

DEPLOY

Solo **2 dipendenze** di runtime: **react** e **react-dom**. Niente router, niente SDK Supabase: tutto il resto è scritto a mano.

Come è fatta l'architettura

CLIENT

React SPA

7 viste, 3 modali
AuthContext per la sessione
Stato in App.jsx

→

src/api

Service layer

supabase.js · library.js
social.js · storage.js
manga.js

→

Supabase

Auth · PostgreSQL + RLS · Storage

Vercel /api/totals

funzione serverless: MangaDex · Kitsu · Jikan + cache

AniList → Kitsu

ricerca e tendenze, con fallback

REGOLA

I componenti non chiamano mai **fetch** direttamente: passano sempre dal service layer, così la logica di rete resta in un solo posto.

Scelte e trade-off

SCELTA	PERCHÉ	COSA COSTA
Supabase invece di Flask	PostgreSQL, Auth JWT e Storage già pronti: il tempo va su modello dei dati e policy RLS.	La sicurezza è nelle policy del database, non in un server scritto a mano.
Funzione serverless	api/totals.js aggrega tre API, valida l'input e mette in cache sulla CDN.	Avvio a freddo; in sviluppo serve vercel dev, altrimenti fallback dal client.
Client REST a mano	Ogni richiesta HTTP è visibile; solo react e react-dom come dipendenze.	Niente realtime né helper dell'SDK ufficiale.
Niente router	App a schede: basta lo stato in App.jsx.	Le viste non hanno un URL proprio.

Struttura del codice

src/

```
App.jsx          // stato globale + tab
main.jsx         // entry point
api/             // chiamate di rete
components/
  common/        // Header, BottomBar…
  views/         // 7 schermate
  modals/        // 3 finestre
context/         // AuthContext
i18n/            // IT / EN
styles/          // CSS + variabili
```

Separazione delle responsabilità

Rete in **api/**, interfaccia in **components/**, sessione in **context/**.

Tema con variabili CSS

Colori in **--bg**, **--red**, **--ink**: cambiare palette significa cambiare poche righe.

Configurazione fuori dal codice

URL e chiave Supabase in **.env**, ignorato da Git.

Navigazione senza router

App.jsx

```jsx
const [activeTab, setActiveTab] = useState('home');
const [selectedManga, setSelectedManga] = useState(null);
const [isDeleteOpen, setIsDeleteOpen] = useState(false);

// la BottomBar cambia solo lo stato
<BottomBar active={activeTab}
           onChange={setActiveTab} />
```

Un'unica pagina: la tab attiva è uno **stato React**. Le schermate e le modali compaiono o spariscono in base allo stato, senza ricaricare nulla.

Home | Libreria | Cerca | Social | Impost.



Autenticazione in 4 vignette

1
Login
Email e password a **/auth/v1/token** con grant_type=password.

2
Sessione
Access e refresh token salvati in **localStorage**.

3
Riavvio
All'apertura il **refresh_token** rinnova la sessione in automatico.

4
Ogni richiesta
setTokenProvider aggiunge l'header Authorization: Bearer.

SICUREZZA Nel client c'è solo la chiave **anon** pubblica: l'accesso ai dati è protetto dalle policy **RLS** di PostgreSQL.

Un client REST scritto a mano

api/supabase.js

```
export async function api(path, opts = {}) {
  const headers = { apikey: SB_KEY, "Content-Type": "application/json" };
  const token = getAccessToken?.();
  if (token) headers.Authorization = `Bearer ${token}`;

  const res = await fetch(`${SB_URL}${path}`, { ...opts, headers });
  if (res.status === 204) return null;
  const data = await res.json();
  if (!res.ok) throw new Error(data.message);
  return data;
}

export const q   = (t, qs) => api(`/rest/v1/${t}?${qs}`);
export const ins = (t, d)  => api(`/rest/v1/${t}`, { method: "POST", ... });
```

- **q** — GET · legge righe
- **ins** — POST · inserisce
- **upd** — PATCH · aggiorna
- **del** — DELETE · elimina

Un CRUD completo in **60 righe**, senza installare l'SDK di Supabase.

Il database PostgreSQL

TABELLA PRINCIPALE

library

id · user_id → profiles
title · cover · genre
status (5 scaffali)
chapters / total_chapters
volumes / total_volumes
score · favourite · review

profiles

Username, avatar, colore. Un profilo ha molti manga.

activity

Feed: aggiunte, voti, completamenti.

follows

follower_id → following_id.
Relazione N:N tra utenti.

recommendations

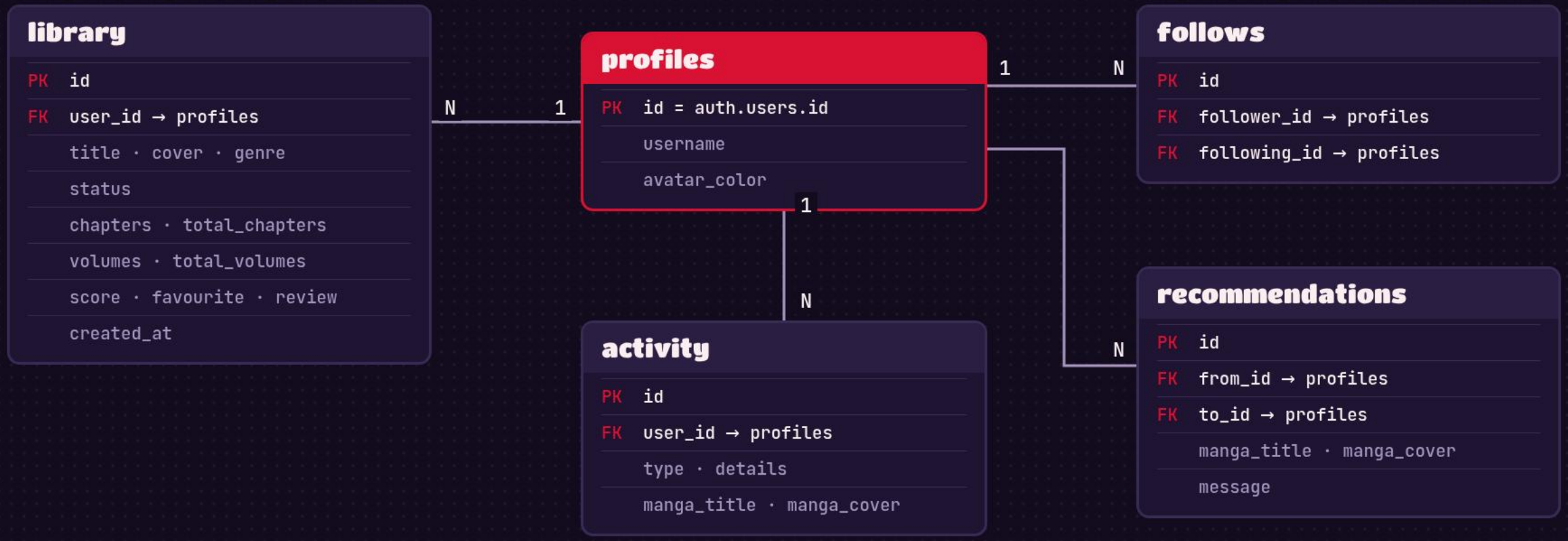
from_id → to_id, manga e messaggio.

Da leggere | In lettura | In pausa | Completati | Abbandonati

MANGATRACK
LA TUA LIBRERIA MANGA

Modello dei dati

Diagramma entità-relazione: ogni riga è legata al profilo dell'utente, protetta da policy RLS.



follows e recommendations collegano due profili: relazione N:N tra utenti

API esterne con piano B

AniList
timeout 6 s · 3 tentativi

→

Kitsu
fallback · 8 s

Se AniList fallisce, riprova con attesa crescente (1 s, 2 s) e poi passa a **Kitsu**.

CAPITOLI E VOLUMI TOTALI

MangaDex | Kitsu | Jikan | 2,5 s max

Dati "in più": se un servizio è lento viene saltato, l'app non si blocca.

api/manga.js

```
async function timedFetch(url, options, ms) {
  const controller = new AbortController();
  const timer = setTimeout(
    () ⇒ controller.abort(), ms);
  try {
    return await fetch(url, {
      ...options,
      signal: controller.signal
    });
  } finally {
    clearTimeout(timer);
  }
}
```

Aggiungere un manga senza attese

1 AWAIT

Insert subito

La riga in **library** si crea con i dati già noti dalla ricerca. È l'unico passo che l'utente aspetta.

2 void

Feed in background

La riga **activity** parte senza await: se fallisce, l'aggiunta resta valida.

3 .then

Arricchimento

Capitoli e volumi totali arrivano dopo e aggiornano la riga con un **PATCH**.

Prima l'aggiunta aspettava 3 API. Ora è istantanea!

Due lingue a runtime

L'app è scritta in **italiano**. L'inglese viene prodotto al volo scambiando le stringhe nel DOM con un dizionario.

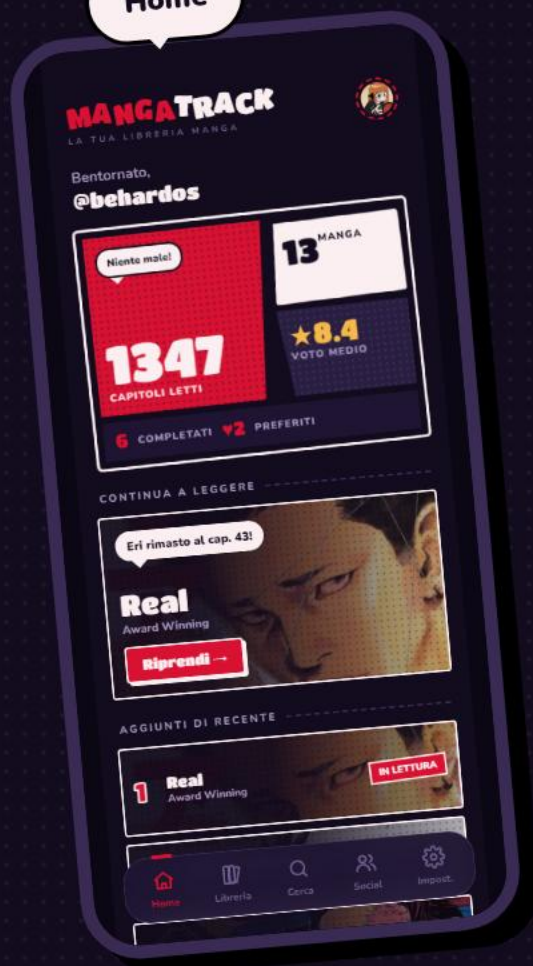
MutationObserver
Traduce anche il testo che compare dopo, come toast e stati vuoti.

localStorage
La scelta resta salvata nella chiave **mt-lang**.

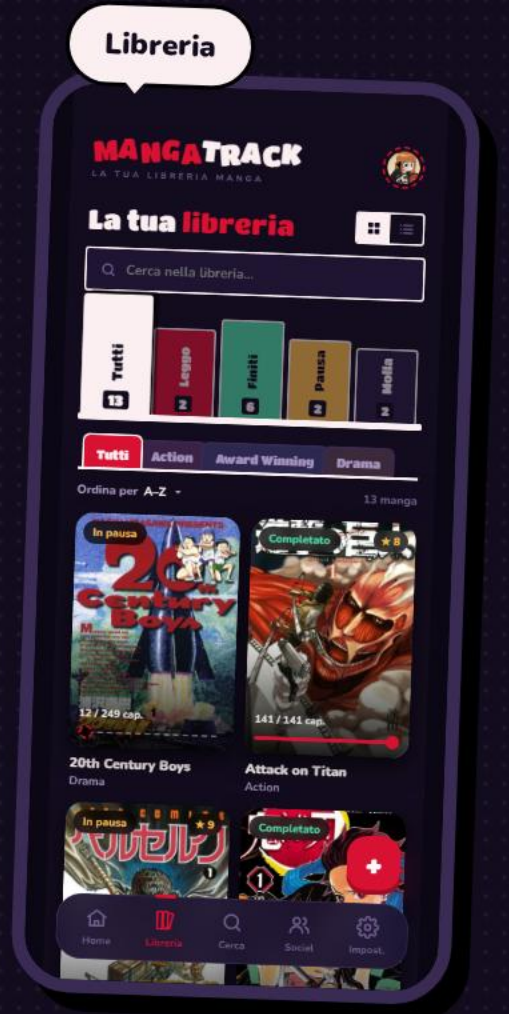


L'app in azione

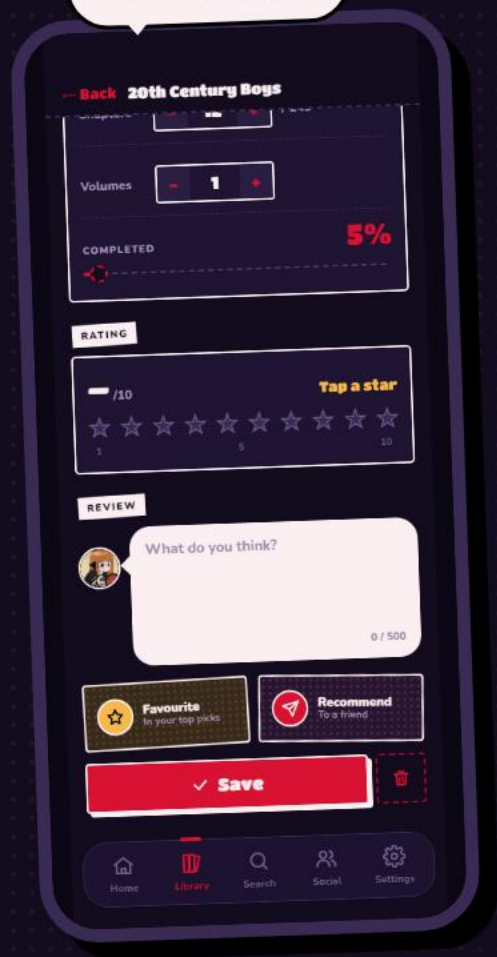
Home



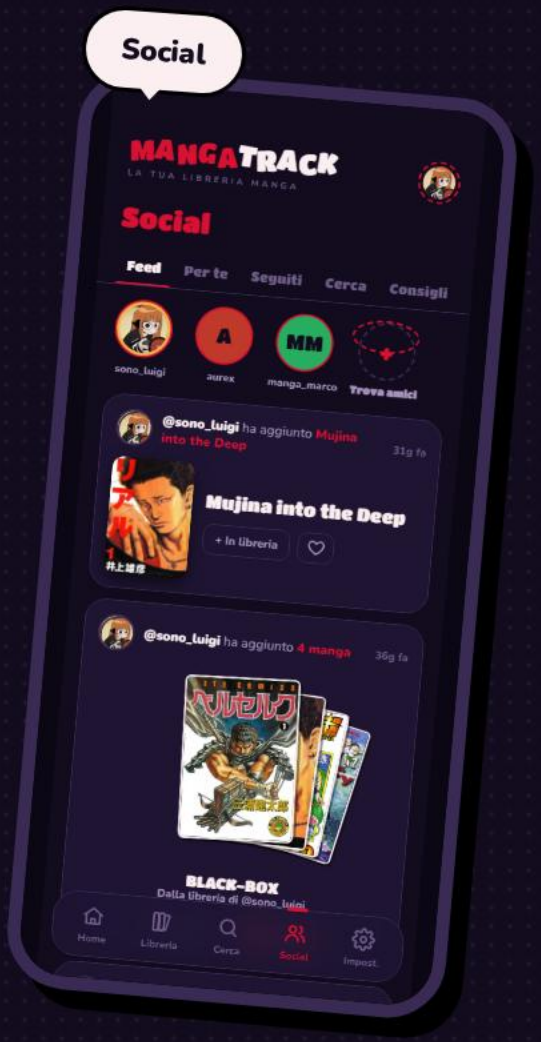
Libreria



Scheda manga



Social



Dal codice al deploy

terminale

```
# sviluppo, porta 3000
npm run dev

# build di produzione in dist/
npm run build

# variabili lette in fase di build
VITE_SUPABASE_URL=...
VITE_SUPABASE_ANON_KEY=...
```

1 **Push su GitHub**
Vercel importa il repo con preset Vite.

2 **Build bloccata se manca .env**
vite.config.js ferma la build senza le variabili Supabase.

3 **Redirect configurati**
Site URL su Supabase per i link di conferma e reset.

Limiti e sviluppi futuri

LIMITI ATTUALI

Nessun test automatico
Le verifiche sono state fatte a mano.

Dipendenza da API esterne
Se AniList e Kitsu non rispondono, la ricerca si ferma.

Traduzione sul DOM
Ogni nuova stringa va aggiunta al dizionario.

Serve la connessione
Senza rete la libreria non si carica.

SVILUPPI FUTURI

01

Test
Vitest e React Testing Library

02

Notifiche
Avviso quando esce un nuovo capitolo

03

Modalità offline
Cache locale della libreria

04

Import
Lista da AniList o MyAnimeList

Cosa ho imparato

Un progetto completo, dal database alla pubblicazione.

01

Frontend

React 18 con hook e Context, navigazione a schede senza router, stato condiviso.

`src/context/AuthContext`

02

Backend e dati

PostgreSQL su Supabase con policy RLS e una funzione serverless su Vercel.

`profiles · library · follows`

03

Integrazione

Client REST scritto a mano e API esterne con timeout, retry e fallback.

`api/supabase.js`

04

Rilascio

Build Vite, variabili VITE_* verificate, deploy continuo su Vercel.

`manga-track-ios.vercel.app`

MANGATRACK
LA TUA LIBRERIA MANGA

L'app in funzione

1 **Home** statistiche e lettura in corso

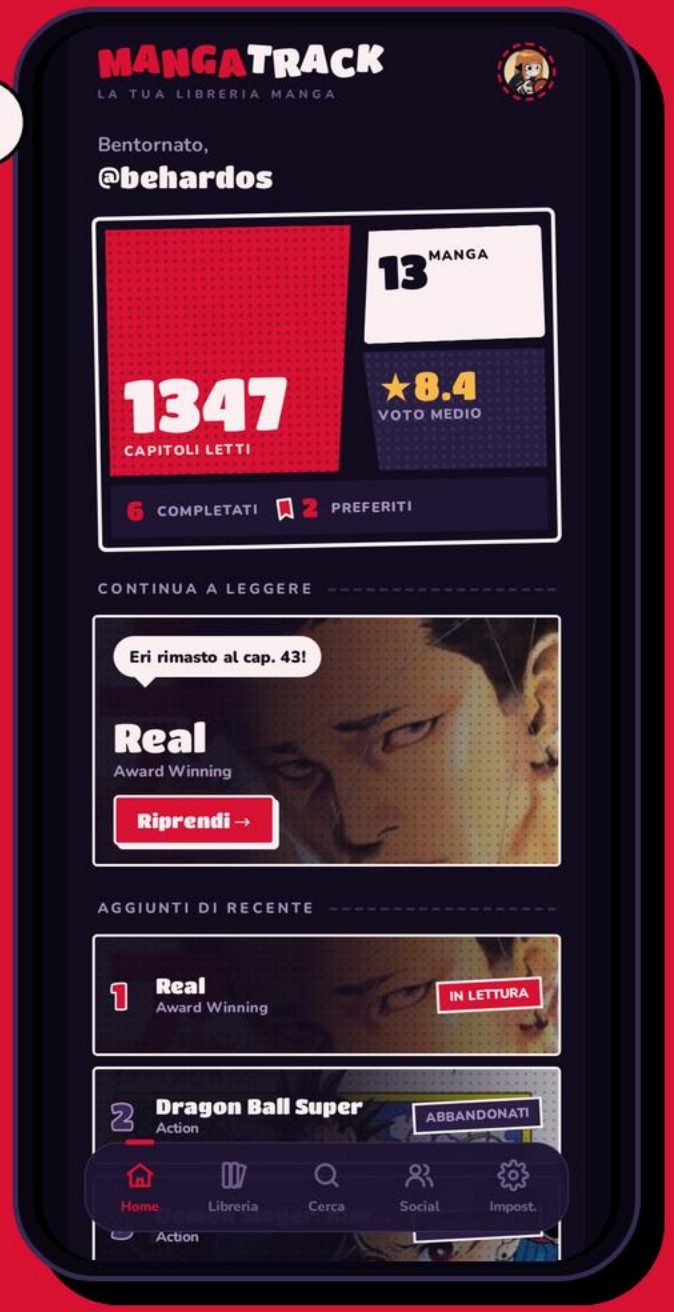
2 **Libreria** scaffali, filtri e copertine

3 **Cerca** trending live da AniList

4 **Social** feed degli amici e consigli

5 **Impostazioni** profilo, lingua e account

Registrato dall'app vera | Modalità demo | Dati live AniList



MANGATRACK

Grazie!

manga-track-ios.vercel.app

